CS781 Capstone Project: In-House Legal Intelligence System

Abhay Kesarwani
240026
abhayk24@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

A law firm cannot send confidential case files to outside AI services, so we built a self-hosted seven-phase legal NLP system for Indian judgments, with no pretrained language model, outside data or paid API. In Phase 1 we trained our own byte-level BPE tokenizer (32,768 tokens) and four Transformer language models from scratch. The best, a 107.8M-parameter Llama-style decoder, saw 1.05B tokens from the four official sources and then 500M more tokens that are 81% Supreme Court text. Phases 2–7 (named entities, rhetorical roles, judgment prediction, statute identification, summarization, prior case retrieval) all start from this model and adapt it with methods we implemented, including the compulsory LoRMA and knowledge distillation. Main findings: the Phase-1 model transfers well to every task; small adapters such as LoRMA match or beat full fine-tuning on small datasets; and for long judgments, choosing which part to read matters more than the fine-tuning method.

1 Competition Result

Codabench username: a_240026 **Roll number:** 240026

Table 1 gives our *best valid* score and rank per phase. Code, notebooks, best submissions and a detailed README: https://github.com/abhayk24/CS781-Legal-Intelligence-System.

Table 1: Official phase scores and ranks. Ranks were recorded on 9 November 2026.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore F1 (0.8412)	13	8
2	Legal NER	Strict macro-F1 (0.6410)	9	15
3	Rhetorical role prediction	Macro-F1 (0.5550)	26	14
4	Court judgment prediction	Macro-F1 (0.7401)	56	8
5	Legal statute identification	Macro-F1 (0.4107)	30	9
6	Abstractive summarization	BERTScore F1 (0.8476)	31	7
7	Prior Case Retrieval (PCR)	Micro-F1@7 (0.4566)	28	5

Total valid Codabench submissions across all phases: 66 **Date ranks were recorded:** 9 November 2026

We met the minimum of 4 distinct submissions in every phase (5 to 15 per phase); each counted submission comes from a different model or method. A file uploaded twice by mistake (once in Phase 2, once in Phase 7) is counted once, and two Phase-5 uploads that never got a score are not counted.

2 Introduction and Problem Description

Law firms often cannot send private documents to cloud AI tools, so the capstone asks for a legal assistant that runs fully in-house: our own small language model trained on public Indian legal text, then taught six legal tasks (Table 2; Phases 2–7 come from IL-TUR [1]) on free GPUs, using only the official course data [2].

Table 2: The seven tasks: input, output and primary metric.

Ph.	Task	Input → output	Primary metric
1	LM pretraining	512 words of a judgment → the next 128 words	BERTScore F1 [3]
2	Legal NER	judgment → list of (start char, end char, label); 12 labels	strict macro-F1
3	Rhetorical roles	list of sentences → one of 13 roles per sentence	macro-F1
4	Judgment prediction	case text → outcome 0 or 1	macro-F1
5	Statute identification	case text → set of statute IDs (0–99)	macro-F1
6	Summarization	judgment → free-text summary	BERTScore F1
7	Prior case retrieval	query case + candidate pool → 7 ranked cases	Micro-F1@7

Constraints:- The Phase-1 model must have 50M–300M parameters and be trained from scratch; no pretrained language model, outside legal corpus or commercial API is allowed; GPU sessions last at most 12 hours. Metrics are strict: in Phase 2 an entity counts only if both offsets and the label are exact; Phase 5 is multi-label over 100 statutes; Phases 1 and 6 use BERTScore. In Phases 2–7, LoRMA [4] is compulsory and knowledge distillation (KD) is mandatory. **Research questions:-** We asked which architecture learns legal text best at the same budget, and whether more data and domain-adaptive pretraining (DAPT) [5] help. We then asked whether the Phase-1 model transfers to the later tasks and which fine-tuning method transfers it best, how a 1,024-token model should read judgments of 5,000–10,000 tokens, and how much decoding, thresholds and ensembles add. **Scope:-** We built every phase end to end, from cleaning and tokenizer to six task models, our own fine-tuning code, local scoring and valid submissions. We did *not* try models above 118M parameters, cross-encoder re-rankers, or chaining phases.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

Phase 1 has four official sources. We kept all Supreme Court and Acts text and a sample of the two huge sources (35% of Tribunal and 21% of High Court documents): about 1.32B training tokens (Supreme Court 250.7M, Acts 148.8M, Tribunals 467.0M, High Courts 454.6M). Each document goes to train, validation or dev by a hash of its ID (seed 1234), so no document is in two splits. Our dev set copies the test format: 479 held-out Supreme Court documents, each giving a 512-word prompt and its true next 128 words. Table 3 covers Phases 2–7.

Table 3: Splits (train / dev / test) for Phases 2–7, and our own held-out sets.

Ph.	Train / dev / test	Notes
2	70 docs (8,393 entities) / – / 35 docs	we hold out 10 docs (label mix matched to the full set); docs average 6,500 tokens
3	80 docs (16,845 sent.) / 10 / 10 docs	40 CL + 40 IT documents for training; dev has only 2,142 sentences
4	≈32,300 / 994 (497 + 497) / test	only 9 of the 994 dev cases fit in 512 tokens
5	≈42,700 / 10,181 / test	100 statute IDs; 43 statutes have over 1,000 training cases
6	7,028 docs / – / 100 docs	we hold out 200 docs (6,828 left for training); reference summaries average 764 words
7	827 q., 4,320 cand. / 118 q., 1,023 / 237 q., 1,727	6.4 (train) and 8.2 (dev) relevant cases per query

3.2 Corpus statistics that actually informed design

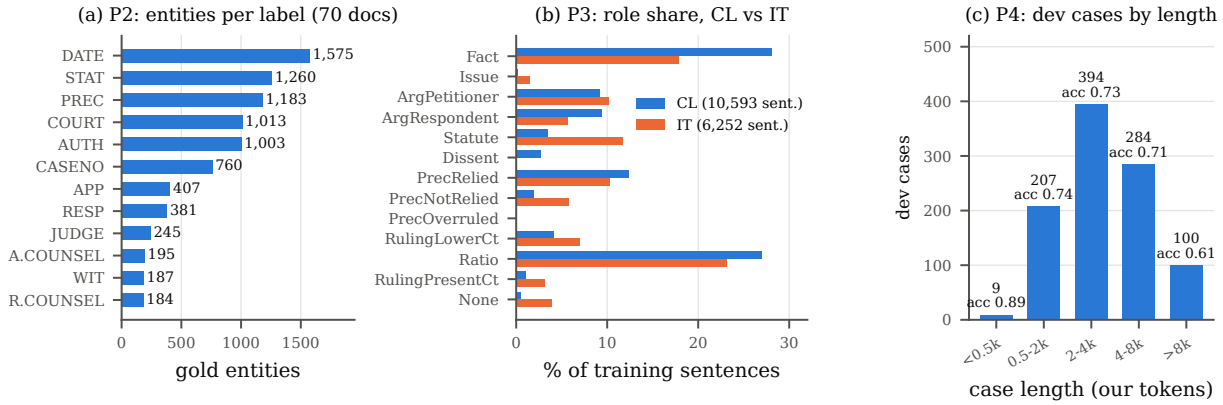


Figure 1: Our own data analysis. (a) Phase-2 entity counts are very uneven. (b) Phase-3 role shares differ between domains; *Dissent* appears only in CL and *PrecedentOverruled* only 3 times in IT. (c) Phase-4 dev cases by length; accuracy of our best round-1 model falls from 0.74 (0.5–2k tokens) to 0.61 (over 8k tokens).

P1:- Test prompts are plain ASCII, single-spaced, and taken from the middle of Supreme-Court-style judgments. So we cleaned all training text the same way, kept all Supreme Court text and gave it two passes (46.6% of training chunks). The sources differ: after main training, validation loss was 2.37 on Supreme Court text but 1.61–1.94 on the other three, so the test-like source was also the hardest. **P2:-** (Fig. 1a). WIT and both COUNSEL labels have under 200 examples, so we weighted the loss by $\sqrt{1/\text{freq}}$. The gold data has 0 overlapping spans and 0 offset errors. A repeated string has the same label 79–98% of the time, which led to document-level label propagation. **P3:-** (Fig. 1b). Fact and RatioOfTheDecision are half of all sentences. Dissent and PrecedentOverruled never occur in the 10 dev documents, so no model can score above 11/13 = 0.846 on dev. CL documents are longer (265 sentences) than IT documents (156); sentences have about 40 tokens in both. **P4:-** (Fig. 1c). Classes are balanced (497/497 on dev), cases are long (384 of 994 exceed 4,096 tokens), and the outcome is usually written near the end. **P5:-** A dev document has 3.92 statutes on average; statutes have 272 to 11,730 training cases and often come in pairs (5 and 34 appear together in 912 dev cases), which led to positive-class weights and one threshold per statute. **P6:-** Documents average 5,088 tokens (median 3,656) and references 994 tokens (73% exceed 512), a compression of about 5×. References reuse much of the judgment's wording: copying its most central sentences already scores 0.85 BERTScore. **P7:-** Queries and candidates have similar lengths

(train mean 8,525 vs. 7,880 tokens, medians 5,655 vs. 4,550; dev means about 10,500 for both). A query has 1 to 92 relevant cases (dev median 5) but we must return exactly 7. About 98–100% of cases mention a year, and a prior case must be older than the query, which gave our "year rule".

3.3 Preprocessing

Cleaning and tokens:- All phases use the Phase-1 cleaning (Unicode NFKC, ASCII quotes and dashes, other non-ASCII removed, whitespace collapsed). We did *not* lowercase, because case carries meaning ("Section 302", party names). Our byte-level tokenizer can encode any string, so citations, section symbols and names are never "unknown". **Character offsets for L-NER:-** We keep a map from every cleaned character to its original position, so every token knows its original span. We tested the alignment by turning gold spans into BIO tags and back: the first test scored only 0.986 (PREC 0.932), because BPE joins punctuation (e.g. "`).`") and a boundary fell inside a token. Splitting punctuation-only tokens into single characters removed most of this loss; the rest are entities ending inside a word ("Railway Board" in "Boards"). **Sentences (P3):-** We used the official sentence lists unchanged, so the order always matches. **Long documents (P4–P6):-** We cut documents into 512-token chunks and compared which parts to read (Section 4). **P7:-** Query and candidate cases are encoded the same way by one shared encoder: up to 8 windows of 512 tokens spread over the whole case, with no special tokens; BM25 features use the full cleaned text.

4 Approach and Model Evolution

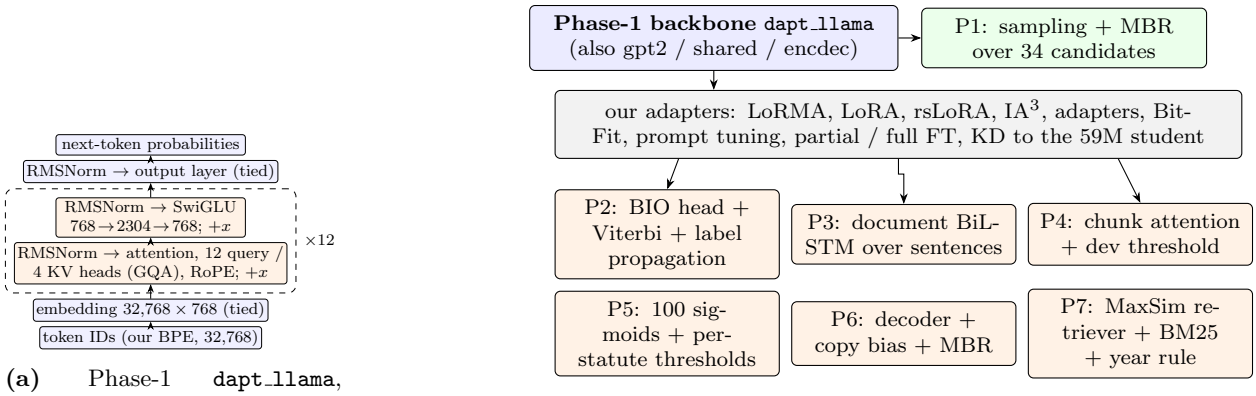
4.1 Phase-1 foundation model

Architecture:- We wrote four very different language models in one PyTorch file, all with hidden size 768, 12 heads, a 1,024-token context, pre-norm blocks and tied embeddings (Table 4). Each saw the same 100M tokens in the same order with the same optimizer, so only the architecture differed. The Llama-style model [6] won clearly (dev perplexity 19.3 vs. 26.3 for GPT-2 style [7]). We then trained it from random weights on 1.05B tokens (`main_llama`, perplexity 9.09). Finally, we continued training on 500M tokens that are 81% Supreme Court text, with a 3× lower learning rate (`dapt_llama`, 8.47). Fig. 2a shows the final model.

Table 4: The four Phase-1 architectures (parameters counted as `sum(p.numel())`).

	GPT-2 style	**Llama style (final)**	Shared	Encoder–decoder
Parameters	111.01M	**107.76M**	58.73M	117.20M
Layers	12	12	6 blocks used twice	6 enc. + 6 dec.
Positions / norm	learned / LayerNorm	RoPE [8] / RMSNorm	RoPE / RMSNorm	RoPE / RMSNorm
MLP (hidden)	GELU (3,072)	SwiGLU [9] (2,304)	SwiGLU (3,072)	SwiGLU (2,304)
KV heads	12	4 (GQA [10])	1 (multi-query)	4 + cross-attn.
Dev loss / ppl (100M)	3.270 / 26.3	**2.962 / 19.3**	3.759 / 42.9	3.582 / 35.9

Size Checks:- `train.py` counts all parameters (tied embeddings once) and stops with an error outside 50M–300M; the final model has 107.76M. The Shared model uses ALBERT-style factorized embeddings and layer sharing [11]. **Tokenizer:-** We trained our own byte-level BPE [12] with a vocabulary of 32,768 = 2^{15} (16 special + 32,752 learned) on a sample weighted towards Supreme Court text, so every ID fits in 2 bytes. Encoding and decoding all 1,000 test prompts returns the exact text. Words like "appellant" are single tokens; rare citations and names are split into pieces but never lost. **Training:-** The objective is next-token prediction on 1,024-token blocks. Passes per source: Supreme Court 2, Acts 0.6, Tribunals 0.65, High Courts 0.4 (main run); Supreme Court



(a) Phase-1 `dapt_llama`, 107.76M.

(b) Whole system: one backbone, six task heads trained separately.

Figure 2: Architecture diagrams.

2 and 0.1–0.15 for the rest (DAPT, 81.4% Supreme Court). In total the model saw 1,049,886,720 + 500M ≈ 1.55B tokens. **Decoding at submission time:-** The model reads the last 824 prompt tokens and writes up to 200 tokens with a KV cache. We use temperature 0.5, top-p 0.9 and repetition penalty 1.1 (the best of 9 settings on dev) and no top-k. Special tokens are blocked, and the text is cut to exactly 128 words (short outputs get extra passes). We then use *MBR (consensus) decoding* [13]: sample K continuations and submit the one that agrees most with the others, where agreement is 0.5·unigram F1 + 0.5·bigram F1 on words. Only our model and word overlap are used; the final submission used $K = 34$. **What failed first:-** Greedy decoding looped (distinct-2 only 0.30), so repetition control was needed; the encoder–decoder and Shared models learned much more slowly. DAPT made Tribunal text slightly worse (validation loss 1.930 → 1.972), a trade worth making for a Supreme-Court-style test.

4.2 Transfer protocol (Phases 2–7)

Every task model starts from a Phase-1 checkpoint (except one random-init ablation in Phase 6) and adds a small head (P6 reuses the LM head; P7 needs no head). We implemented all methods ourselves in plain PyTorch (no PEFT library): full and partial fine-tuning (top 4 of 12 layers), frozen backbone + head, BitFit [14], LoRA [15], rsLoRA [16], Houlsby [17] and Pfeiffer [18] adapters, IA^3 [19], prompt tuning [20] and KD [21]. **LoRMA** *multiplies* a frozen weight W_0 by a low-rank change:

$$W' = \Big(I + \tfrac{\alpha}{r} BA\Big) W_0, \qquad y = W_0 x + \tfrac{\alpha}{r}\, B\big(A\,(W_0 x)\big), \tag{1}$$

with $r = 16$, $\alpha = 32$, A random and $B = 0$ at the start, so training starts exactly at the Phase-1 model; it is applied to q, k, v, o (P6, P7, P3 round 4: also gate, up, down). **KD**: a fine-tuned llama teacher (107.8M) and the Shared student (58.7M), with $\mathcal{L} = 0.5\,\mathcal{L}_{\text{task}} + 0.5\,T^2\,\mathrm{KL}(p_{\text{teacher}} \,\|\, p_{\text{student}})$ on outputs softened with $T = 2$ (P5: BCE to the teacher's softened probabilities; P7: KL to the teacher's similarity distribution over the batch, $T = 1$). In P2–P5 decoders run without the causal mask (except GPT-2, which broke without it) and the encoder–decoder uses only its encoder; P6 and P7 keep the causal mask. Fig. 2b shows the whole system.

4.3 Phase 2 - Legal NER

We use token classification, then rebuild exact character offsets with the map from Section 3.3. The backbone feeds dropout 0.1 and a linear layer over 25 BIO tags. Training cuts each document into 512-token windows from a random start; prediction uses 1,024-token windows (step 512), weighting

tokens near a window's center more. Decoding is a *constrained Viterbi* (I-X may only follow B-X or I-X), followed by *label propagation*: other copies ($\geq$4 characters) of a predicted string in the document get the same label, only for labels where this helped on validation. Imbalance is handled by loss weights $\sqrt{\#\text{O}/\#\text{tag}}$, clipped to $[1, 10]$.

4.4 Phase 3 - Rhetorical role prediction

The judgment is cut into 512-token windows (at test time, two cuttings shifted by 256 tokens are averaged), and token vectors are averaged per sentence. A 2-layer BiLSTM reads *all sentences of the document in order*, with each sentence's position and a CL/IT flag, and labels every sentence. Both domains are trained together. Rare roles get class weights, and each role's probability is divided by its frequency$^{\tau}$ (τ chosen on dev). Viterbi role transitions were rejected (Table 6).

4.5 Phase 4 - Court judgment prediction

A case is read as chunks (512 tokens; 1,024 in round 2); a chunk vector is the mean of its token vectors, and learned attention pooling ($768 \to 128 \to 1$) builds the case vector for a $768 \to 2$ classifier. Of five readings compared, the final models read the *last 512 or 1,024 tokens*. The threshold on $P(y{=}1)$ is chosen on dev (0.05–0.95, step 0.01) and reused on test. Phase-2/3 outputs were not used.

4.6 Phase 5 - Legal statute identification

Up to 4 chunks of 512 tokens are spread over the whole document; attention pooling (or the max over chunk scores) feeds 100 sigmoids whose bias starts at the log-odds of each statute's training frequency. The loss is BCE with positive weights $\sqrt{\text{neg}/\text{pos}}$ in $[1, 30]$ (focal loss [22] also tried). Thresholds: 0.5, one global value, or one per statute (global below 5 dev cases); per-statute won.

4.7 Phase 6 - Abstractive summarization

Our main model (llama) is decoder-only. The input is `[document] " Summary:" [summary]`, and the loss is on the summary only (the encoder–decoder reads up to 896 tokens). Instead of chunk-then-merge we *select* what to read: the first tokens (`lead`), the most central sentences by TF-IDF similarity to the document, in order (`central`), or half of each; round 3 reads 768 tokens and writes up to 254. Length and repetition: greedy decoding, no repeated 4-grams, at least 80% of the maximum length. A *copy bias* adds +2 to the score of every token that occurs in the input; we keep the consensus of 8 candidates (1 greedy + 7 sampled), and finally of 5 outputs of two LoRMA llama models (`ens_mbr5`). No preference training or RL was used.

4.8 Phase 7 - Prior Case Retrieval (PCR)

Our system is a sparse+dense hybrid with one shared bi-encoder. A window vector is the unit-length mean of the final hidden states; cases are compared by *MaxSim* [23] (each query window takes its best candidate window; then averaged) or by the cosine of mean vectors. Training uses InfoNCE [24] (temperature 0.05): in each step 8 queries must rank their relevant window above 15 others (the other 7 relevant windows and 8 BM25 hard negatives), with the query's other relevant cases masked. The final score is a weighted sum of z-scored signals: BM25 [25] on words and on word pairs (k_1, b set on train), our retriever, the year rule (a later case cannot be a prior case), a penalty for generic "hub" cases and TF-IDF feedback; weights are chosen on dev by coordinate

ascent with a 2-fold check, and the retriever always keeps a non-zero weight. **Failed:** TF-IDF feedback was never chosen; KD helped the 59M student only a little (0.280 vs. 0.270 alone), far behind the teacher (0.375).

5 Experiments and Implementation Details

5.1 Compute and training protocol

All runs used Kaggle, 2× T4 (16 GB). Phase-1 used both GPUs with DistributedDataParallel: `main_llama` took about 8.2 hours (4,005 steps of 262,144 tokens) and DAPT about 3.3 hours. In Phases 2–7 we ran one experiment per GPU with a 25–90 minute budget, about 140 task runs in total. We used AdamW (P1: $\beta = (0.9, 0.95)$, weight decay 0.1; later 0.01), 4–6% warm-up, cosine decay, clipping 1.0, fp16. Early stopping keeps the best checkpoint by dev continuation loss (P1), validation strict macro-F1 (P2), dev macro-F1 (P3–P5), validation loss (P6) or quick dev Micro-F1@7 (P7).

Table 5: Hyperparameters of the final model in each phase. Dropout "0.1 / 0.05" = head / adapter; P1 also uses repetition penalty 1.1.

Hyperparameter	P1	P2	P3	P4	P5	P6	P7
Parameters (M)	107.76	107.8	111.5	108.6	≈108	107.8	107.8
Context length	1,024	512 / 1,024 test	512	last 1,024	4 × 512	768 + 254	8 × 512
Peak LR	2e-4 (main 6e-4)	1e-3	3e-4	5e-4	5e-5	5e-4	3e-4
Batch × accum.	8 seq. × 16 (2 GPUs)	16 windows	1 document	16 chunks	4 docs × 4	8 sequences	8 queries
Epochs / steps	4,005 + 1,907 st.	best ep. 7	best ep. 15	3–4 epochs	1 + 2 passes	≤30 min	best ep. 18
Dropout	0.0	0.1 / 0.05	0.05	0.1 / 0.05	0.1	0.05	0.05
Decoding / threshold	MBR 34, T 0.5, top-p 0.9	Viterbi + propag.	prior corr. τ	dev threshold	per-statute	copy + MBR	top 7, fused
Method (trainable M)	DAPT (all)	LoRMA (0.81)	LoRMA (4.48)	LoRMA (0.89)	partial (27.6)	LoRMA (2.85)	LoRMA (2.85)

5.2 Local evaluation

Every phase has a held-out split (Section 3.1), and every run was scored with an exact copy of the official script. With large held-out sets, test stayed close: P4 0.7500 vs. 0.7401; P5 1.1–2.2 points lower; P6 0.002–0.004 lower; P7 2-fold dev 0.4525–0.4647 vs. test 0.4507–0.4566. With 10 documents (P2, P3) local scores were noisy: in P3 test was 3–4 points *higher* than dev, partly because two roles missing from dev always count as 0; in P2 a 1,024-token training window raised validation (0.607 → 0.634) but lowered test (0.582 → 0.558).

5.3 Ablations and controlled comparisons

Table 6 changes one factor at a time (local held-out scores; all else fixed). The clear positive results are: more pretraining data and DAPT in P1; constrained Viterbi and label propagation in P2; whole-document context in P3; reading the end of the case in P4; reading the whole document in P5; reading central sentences, reading more and writing shorter summaries in P6; MaxSim pooling in P7; and Phase-1 initialization everywhere (random init lost 0.011 BERTScore in P6). We also report the negative results. Viterbi role transitions in P3 helped dev but hurt test, so we dropped them. The BiLSTM head in P2 (0.552 vs. 0.607), focal loss in P5 (0.393 vs. 0.401) and longer

1,024-token windows in P2 and P3 did not help. KD did not help the summarization student in round 1, and TF-IDF feedback was never chosen in P7. Bidirectional attention broke GPT-2 in P2, because its learned positions only work left to right.

Table 6: One-factor ablations (local held-out scores, primary metric of the phase).

Ph.	Factor changed	Result
2	greedy → Viterbi → + propagation	0.4225 → 0.5260 → 0.6074
2	technique (llama)	LoRMA 0.607, LoRA 0.588, full 0.538, frozen + head 0.228
2	GPT-2: bidirectional → causal	0.217 → 0.442
3	sentence / window / +BiLSTM / document	0.243 / 0.443 / 0.473 / 0.499
3	train on CL only / IT only / both	0.380 / 0.391 / 0.473
3	technique (window model)	LoRMA .473, LoRA .441, rsLoRA .426, IA^3 .424, full .422, head .331
3	Viterbi role transitions	dev +0.008, but lower test (0.5006 vs. 0.5204): rejected
4	read first / first+last / 4 chunks / last	0.567 / 0.672 / 0.672 / 0.685
4	technique (last 512)	partial .716, full .714, adapter .699, LoRMA .696, BitFit .661, head .623
4	threshold 0.5 → tuned on dev (encdec)	0.533 → 0.629
5	read first 512 / last 512 / whole doc	0.366 / 0.368 / 0.401
5	technique (whole doc)	partial 0.401, full 0.394, adapter 0.372, LoRMA 0.358, prompt 0.315
5	loss BCE / focal / weighted; +2 passes	0.393 / 0.393 / 0.401; 0.401 → 0.421
6	technique (llama, lead 512)	Houlsby 0.8343, full 0.8333, LoRA 0.8332, Pfeiffer 0.8331, LoRMA 0.8328
6	Phase-1 weights vs. random init	0.8333 vs. 0.8224 (validation loss 2.01 vs. 3.38)
6	summary 128 / 256 / 510 tokens	0.8370 / 0.8388 / 0.8328
6	lead vs. central; 512 → 768 tokens	0.8328 vs. 0.8371; 0.8420 → 0.8452
7	retriever alone: none/IA^3/full/partial/LoRMA	0.304 / 0.356 / 0.364 / 0.369 / 0.375
7	pooling: mean vs. MaxSim (LoRMA)	0.358 vs. 0.375
2–7	KD: student without → with teacher	P2 .495→.572, P3 .388→.407, P4 .646→.702, P5 .304→.338, P6 .8209→.8188, P7 .270→.280

6 Results

Local-dev and official numbers:- Table 6 gives local scores for every major variant. Table 7 lists at least three variants per phase with the local score next to the Codabench score, so the generalization gap is visible (discussed in Section 5.2). Table 8 gives the official secondary metrics of each final system (the Phase-1 script reports ROUGE-L and BLEU-4, the Phase-6 script ROUGE-1 and ROUGE-L), and Table 9 gives per-class F1 for Phases 2, 3 and 5, because macro-F1 hides weak rare labels.

Table 7: Per-phase variant comparison (at least three per phase): local held-out score and Codabench primary metric. Best per phase in bold; "–" = not scored locally.

Phase	Variant	Local	Codabench	Phase	Variant	Local	Codabench
1	`main_llama`, 1 sample	0.8354	0.8324	4	**ens_greedy, round 2**	0.7500	**0.7401**
1	`main_llama`, MBR 8	0.8424	0.8375	4	ens_top3, round 2	0.7402	0.7310
1	100M models (all 4)	.821–.827	.819–.825	4	llama LoRMA, last 1,024	0.7334	0.7349
1	`dapt_llama`, MBR 12	–	0.8390	4	ens_top5, round 1	0.7360	0.7105
1	**`dapt_llama`, MBR 34**	–	**0.8412**	4	round-1 singles (4 models)	.601–.716	.588–.702
2	**ens_top8 (8 models)**	0.6426	**0.6410**	5	**ens_greedy, round 2**	0.4313	**0.4107**
2	llama LoRMA (best single)	0.6074	0.5819	5	llama partial FT, continued	0.4209	0.4011
2	shared KD	0.5718	0.5317	5	llama LoRMA	0.3575	0.3411
2	gpt2 full FT, causal	0.4416	0.4027	5	shared KD	0.3379	0.3246
2	encdec LoRA	0.3813	0.3462	5	gpt2 LoRMA	0.2871	0.2760
3	**ens_top8, round 5**	0.5126	**0.5550**	6	**ens_mbr5 (5 outputs)**	0.8492	**0.8476**
3	llama LoRMA (best single)	0.5003	0.5204	6	LoRMA, 768 tokens, copy	0.8477	0.8443
3	gpt2 LoRMA	0.4490	0.4625	6	gpt2 LoRMA	0.8297	0.8261
3	shared KD	0.4069	0.4268	6	encdec LoRMA	0.8212	0.8179
3	encdec LoRMA	0.3416	0.3024	6	shared KD	0.8188	0.8168
				7	**llama LoRMA + fusion**	0.4647	**0.4566**
				7	llama IA^3 + fusion	0.4591	0.4530
				7	llama full FT + fusion	0.4647	0.4519
				7	shared KD + fusion	0.4636	0.4519
				7	llama partial FT + fusion	0.4647	0.4507

Table 8: Final systems on the local held-out sets, with the official secondary metrics.

Phase	System (held-out set)	Primary metric; secondary metrics
1	`main_llama`, MBR 8 (479 dev; best scored)	BERTScore 0.8424; ROUGE-L 0.2113, BLEU-4 0.0714
2	`ens_top8` (10 validation docs)	strict macro-F1 0.6426; best single: P 0.600, R 0.651
3	`ens_top8`, round 5 (10 dev docs)	macro-F1 0.5126; CL 0.460, IT 0.497
4	`ens_greedy` (994 dev cases)	macro-F1 0.7500; accuracy 0.751, P(1) 0.775, R(1) 0.706
5	`ens_greedy` (10,181 dev cases)	macro-F1 0.4313; micro-F1 0.5206, subset accuracy 0.0512
6	`ens_mbr5` (200 validation docs)	BERTScore 0.8492; ROUGE-1 0.4069, ROUGE-L 0.2288
7	llama LoRMA + fusion (118 dev queries)	Micro-F1@7 0.4647; P@7 0.506, R@7 0.430

Table 9: Per-class F1 on the local held-out sets (best single model per phase).

Phase / model	Per-class F1
P2, llama LoRMA (validation)	APP .305, RESP .131, A.COUNSEL .741, R.COUNSEL .667, JUDGE .614, WIT .980, AUTH .526, COURT .671, STAT .794, PREC .366, DATE .950, CASENO .544
P3, llama LoRMA lr 3e-4 (dev)	Fact .717, Issue .688, ArgPet .719, ArgResp .674, Statute .769, Dissent .000, PrecRelied .528, PrecNotRelied .339, PrecOverruled .000, RulingLower .466, Ratio .671, RulingPresent .659, None .275
P5, llama partial FT, continued (dev)	43 frequent statutes (>1,000 train cases) mean .504 vs. 57 others .358; rarest third .337; worst: statute 8 .087, 0 .138, 52 .147

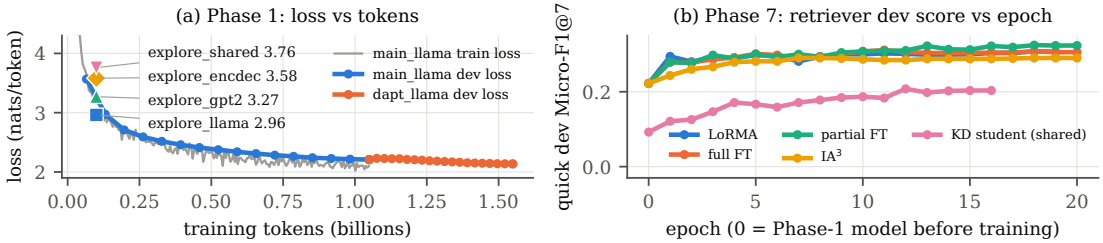


Figure 3: Learning curves. (a) Phase 1: training loss and dev continuation loss of `main_llama` against training tokens, the four 100M-token exploration models, and the dev loss during DAPT (2.207 → 2.136). (b) Phase 7: quick dev Micro-F1@7 (2 windows per case) of the retriever alone after each epoch; best epochs: LoRMA 18, partial 18, full 11, IA^3 9, KD 12.

Learning curves (Fig. 3):- The Phase-1 dev loss is still falling at the end, so more data would help; the full fine-tuned retriever stops improving early, LoRMA and partial keep improving. **Which system won and why:-** Every winner is built on `dapt_llama`, which beat the other Phase-1 models in all six tasks; this is not just "more training", since llama already had the best dev loss at the equal 100M-token budget. LoRMA gave the best single model in Phases 2, 3, 6 and 7: these tasks have small training sets (60–7,000 documents), and LoRMA changes only 0.7–2.6% of the weights, starting at the exact Phase-1 model, so it keeps the legal knowledge and overfits less. With 32,000–43,000 training cases (P4, P5) there is enough data to train more weights, and partial fine-tuning was as good or better. Ensembles added the last 0.3–6 points in Phases 2–6, because their members make different mistakes.

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

P1:- Copying the last 128 prompt words already scores 0.8424 BERTScore on dev, so small gains are close to noise. **P2:-** (Table 9). APP vs. RESP depends on a party's role, not on how the name looks, so these two are the weakest; PREC spans are long citations with brackets, so one-character boundary misses are common. The 10-point gain from greedy to Viterbi shows broken boundaries

were the largest error type. **P3:-** The weakest roles come in related pairs (Table 9): relied vs. not-relied precedents, and lower-court vs. present-court rulings. "The appeal is dismissed." can be either ruling; only its place in the judgment tells which. *None* is weak because headings differ in every judgment. **P4:-** Accuracy falls with length (Fig. 1c; final model 0.79 to 0.67), as the model sees less of long cases. Outcome 1 is missed more often than wrongly predicted (final ensemble: 146 vs. 102). **P5:-** When unsure, the model falls back to frequent statutes. Top confusions (gold missed → other predicted): 14→5 (161 cases), 39→43 (135), 12→5 (131); the pair 43+97 is predicted 1,865 times but is gold only 862 times. **P6:-** Only 70.5% of the numbers (sections, years) in greedy summaries occur in the judgment (81–90% with copy bias), and only 17–27% of the reference's numbers appear, as the model reads only about 15% of an average judgment. **P7:-** Final system on dev: 418 true positives, 408 false positives, 555 false negatives (precision 0.506, recall 0.430). A fixed list of 7 limits recall, and landmark "hub" cases are often wrong picks.

7.2 What the metrics do not measure

BERTScore and ROUGE [26] reward similarity to one hidden reference, not legal correctness: a fluent summary with a wrong section number can score well. A plain extractive baseline (central sentences, no model) scored 0.8500 on our validation set, above our best system (0.8492). Strict span-F1 gives zero to an entity whose boundary is off by one character, and Micro-F1@7 forces exactly 7 answers. In a real firm, a lawyer must still check facts and citations.

7.3 Limitations and what you would do next

Two T4 GPUs limited us to a 108M model, a 1,024-token window and 1.55B tokens. We did not deduplicate the corpus; each architecture had one seed; 10-document held-out sets were noisy; there was no human legal evaluation; phases were trained separately (no error propagation, but no help between phases); and with no outside corpora, rare statutes and roles had few examples. Next, we would deduplicate the corpus and train a 150M model with a longer context, train Phases 4–7 on several windows per document, add a cross-encoder re-ranker in Phase 7, and use Phase-3 roles to choose what Phases 4 and 6 read.

7.4 Surprising findings

(1) Small adapters beat full fine-tuning on small data: in P2, LoRMA with 0.81M trainable parameters beat full fine-tuning of 107.8M by 7 points (0.607 vs. 0.538) with less memory (3.96 vs. 6.45 GB). (2) In P7, BM25 on word pairs alone (0.437) beat every neural retriever alone (best 0.375), because citations follow exact shared phrases; our retriever still added 0.006–0.010 on top.

8 Conclusion

Three choices moved the metrics most. First, the Phase-1 foundation: a fair 100M-token architecture race, then 10× more data plus Supreme-Court-focused DAPT; this model was the best start in every later phase. Second, choosing *what to read* in long judgments (the end for outcomes, the whole document for statutes and roles, central sentences for summaries). Third, decoding and post-processing: MBR, Viterbi with propagation, tuned thresholds, copy bias and ensembles. Section 5.3 lists what did not help. The main lesson: under a parameter and data lock, a compact model trained only on legal text is a strong base for many legal tasks, if it is adapted gently (small adapters, low learning rates) and long documents are handled with care.

LLM Usage Declaration

Completing this section is mandatory. Inconsistent answers relative to the code and the prose will be treated as an integrity issue.

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Claude x / Grok / Copilot / other).

Answer: Claude.

Q2. For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Explanations of LoRMA, KD, BM25, MBR, BERTScore and the task briefs along with lecture note topic to get summary
Ideation / brainstorming at a high level	Yes	Some experiments and techniques to try in each phase
Debugging small ancillary snippets	Yes	Fixing errors shown in Kaggle logs many times
Plotting / visualization code	Yes	In this report figure and diagrams used claude
Grammar / writing style of the report	Yes	For Grammatical check,I upload report in claude to correct it.
Report structure / section wording	Yes	Provide results and recheck everything for verification only.
Designing the model architecture	No	Model design by me himself,only coding part, I took help of claude.
Hyperparameter search / tuning	No	
Analysis of results / conclusions	No	
Writing the core training / inference code	Yes	Most pf the code wrote by claude,as it is too long and idea and understanding is mine only.

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer: Yes. The official CS-781 starter notebook, dataset and evaluation scripts [2]; the IL-TUR benchmark paper [1]; the papers cited in this report for every method we used (Transformer, GPT-2, LLaMA, RoPE, SwiGLU, GQA, ALBERT, BPE, DAPT, LoRMA, LoRA, rsLoRA, Houlsby and Pfeiffer adapters, IA^3, prompt tuning, BitFit, KD, focal loss, MBR, BM25, ColBERT, InfoNCE, BERTScore, ROUGE, BLEU); and the official documentation of PyTorch, Hugging Face `tokenizers` and `huggingface_hub`.

Q4. Apart from any permitted LLM use, what was *your* contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis. Name at least two things you tried that failed and what you concluded from them.

I ran every experiment of this project myself on Kaggle: the data preparation, the three Phase-1 training stages and lots of task runs over Phases 2–7, while managing the weekly GPU quota and the Hugging Face storage. Finds every error, and re-ran the notebooks until all runs and zips were

valid. I studied the methods (LoRMA, KD, MBR, BM25 and the metrics) until I could explain why each one helped or did not help. I chose which models to submit in each phase and in what order, so that every phase had at least four submissions from different Phase-1 models and methods. I checked that the local scores matched the official evaluation scripts and compared every test score with its dev score. One thing that failed was the first Phase-7 notebook: its safety test ran for about four hours because the small test pool had no relevant candidates, so the training loop never ended; I learned to always put limits and timeouts on loops. Another failure was bidirectional GPT-2 in Phase 2 (0.217 on validation): its learned positions only work left to right, so I learned that not every Phase-1 model can be used the same way. A third was Viterbi decoding in Phase 3, which helped on 10 dev documents but lowered the test score; I concluded that a tiny dev set cannot be trusted for small gains. I also learned from Phase 6 that a simple extractive baseline can beat a neural model on BERTScore, so high metric scores do not always mean a better legal summary.

References

[1] A. Joshi, S. Paul, A. Sharma, P. Goyal, S. Ghosh, and A. Modi, "IL-TUR: Benchmark for Indian legal text understanding and reasoning," in *Proceedings of ACL*, 2024.

[2] CS-781 Course Team, IIT Kanpur, "CS-781 capstone: Phase-1 starter notebook, dataset (Exploration-Lab/CS-781-Capstone) and official evaluation scripts," 2026.

[3] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," *International Conference on Learning Representations*, 2020.

[4] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," *arXiv preprint*, 2025.

[5] S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith, "Don't stop pretraining: Adapt language models to domains and tasks," in *Proceedings of ACL*, 2020.

[6] H. Touvron, T. Lavril, G. Izacard, *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[7] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," tech. rep., OpenAI, 2019.

[8] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[9] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[10] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of EMNLP*, 2023.

[11] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A lite BERT for self-supervised learning of language representations," in *International Conference on Learning Representations*, 2020.

[12] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of ACL*, 2016.

[13] S. Kumar and W. Byrne, "Minimum Bayes-risk decoding for statistical machine translation," in *Proceedings of HLT-NAACL*, 2004.

[14] E. Ben Zaken, Y. Goldberg, and S. Ravfogel, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proceedings of ACL*, 2022.

[15] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[16] D. Kalajdzievski, "A rank stabilization scaling factor for fine-tuning with LoRA," *arXiv preprint arXiv:2312.03732*, 2023.

[17] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *International Conference on Machine Learning*, 2019.

[18] J. Pfeiffer, A. Kamath, A. Rücklé, K. Cho, and I. Gurevych, "AdapterFusion: Non-destructive task composition for transfer learning," in *Proceedings of EACL*, 2021.

[19] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Advances in Neural Information Processing Systems*, 2022.

[20] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proceedings of EMNLP*, 2021.

[21] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[22] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of ICCV*, 2017.

[23] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proceedings of SIGIR*, 2020.

[24] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[25] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[26] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, pp. 74–81, 2004.